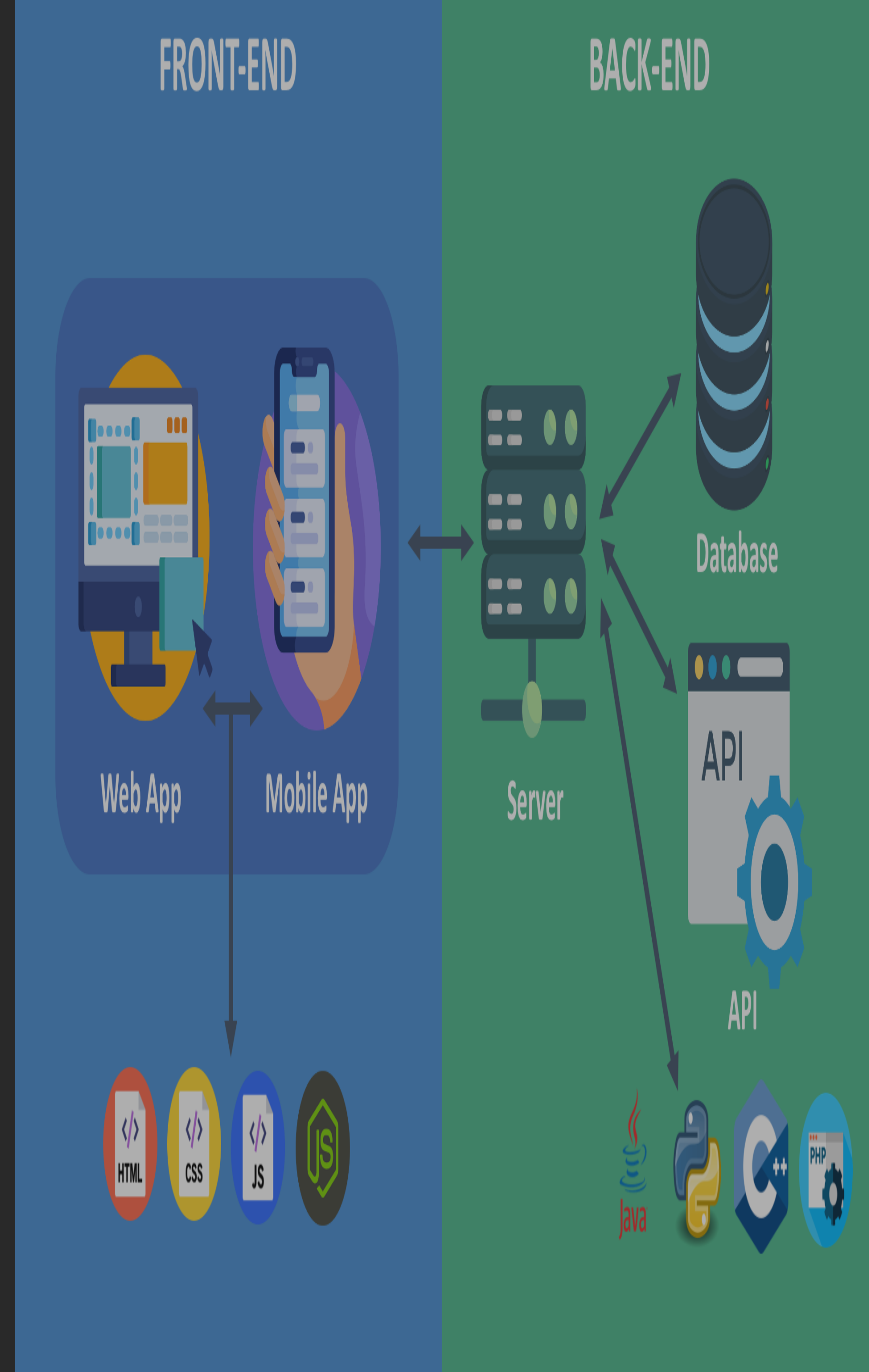


DAY 3

State, Events & Backend

Interactive UI with State Management and API
Communication



What Is State?

State is **data that can change** while the application is running. It enables dynamic, interactive user interfaces.

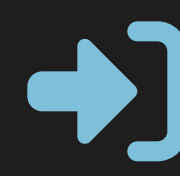
Common State Examples



Counter value



User name



Login status



Shopping cart

When state changes, React **automatically re-renders** the component to reflect the updated data. This is called **reactive rendering**.

useState Hook

Step 1: Import

```
import { useState } from 'react';
```

Step 2: Create State

```
const [count, setCount] = useState(0);
```

How It Works

`useState(0)` — initializes state with value 0

`count` — current state value

`setCount` — function to update the value

The array **destructuring** syntax `[ ]` extracts both values from `useState`.

Display State

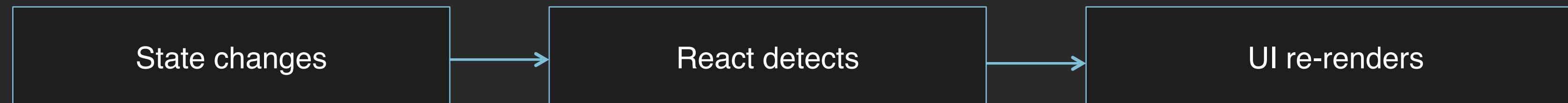
Render State in JSX

```
<h1>{count}</h1>
```

Live Preview

0

How React Updates the UI



Whenever state changes, React **automatically updates the UI**. This is the core of React's reactive programming model — you describe what the UI should look like for each state, and React handles the updates.

Update State

Button with onClick

```
<button  
  onClick={() => setCount(count + 1)}  
>  
  Increase  
</button>
```

Result

Every click increases the counter. State updates trigger re-renders automatically.

Important Rules

Never mutate state directly — always use the setter function.

State updates are **asynchronous** — React batches them for performance.

Each component has its own **independent state**.

Events

React events use **camelCase** syntax and are attached directly to JSX elements.

Common React Events

onClick

Fires when the user clicks an element. Used for buttons, links, and interactive elements.

onChange

Fires when the value of an input changes. Used for text fields, checkboxes, and selects.

onSubmit

Fires when a form is submitted. Used for login forms, search forms, and data entry.

Example

```
<button onClick={handleClick}>
```


Controlled Input

The input value is **controlled by React state** — the single source of truth.

Step 1: Create State

```
const [name, setName] = useState("");
```

Step 2: Bind Input to State

```
<input  
  value={name}  
  onChange={(e) => setName(e.target.value)} />
```

Data Flow

